

Guía de inducción — Sistema de Reservas Bienestar CONAF

Documento de inicio para quien se integra al proyecto `coipo_cabania`. No supone experiencia previa en React ni en desarrollo web: cada paso dice **qué archivo abrir, qué línea cambiar, cómo comprobar que funcionó y qué hacer si falla**.

Tiene dos partes:

1. **15 ejercicios guiados** — desde crear una rama en GitHub Desktop hasta depurar un error. Se hacen en orden; cada uno introduce un concepto nuevo usando la app real.
2. **Una funcionalidad completa de principio a fin** — sistema de valoración con 5 estrellas, desde crear la rama hasta abrir el Pull Request.

Regla de oro del proyecto: algo está "listo" solo cuando **viste el resultado funcionar en el navegador**. Que el código compile no significa que funcione.

Índice

- [Parte 0 — Preparación del ambiente](#)
- [Mapa del proyecto](#)
- [Parte 1 — 15 ejercicios guiados](#)
- [Parte 2 — Funcionalidad completa: valoración con 5 estrellas](#)
- [Anexo A — Glosario](#)
- [Anexo B — Errores frecuentes y su significado](#)

Parte 0 — Preparación del ambiente

Solo se hace una vez.

0.1 Instalar los programas

Programa	Para qué	Dónde
Node.js 22 LTS	Ejecuta el proyecto (trae <code>npm</code>)	https://nodejs.org
GitHub Desktop	Manejar ramas, commits y PR sin usar la consola	https://desktop.github.com
Visual Studio Code	Editar el código	https://code.visualstudio.com
Google Chrome	Probar la app y usar las herramientas de desarrollo	https://google.com/chrome

Extensiones recomendadas en VS Code (pestaña de extensiones, ícono de cubos): `ESLint`, `Tailwind CSS IntelliSense`, `Prettier`.

0.2 Clonar el repositorio

1. Abre **GitHub Desktop** → **File** → **Clone repository...**
2. Pestaña `GitHub.com` → busca `Sud-Austral/coipo_cabania`

3. **Local path**: deja la ruta por defecto (ej. `C:\Users\tu.usuario\Documents\GitHub`)

4. **Clone**

Si no aparece el repositorio, pide que te agreguen como colaborador en la organización **Sud-Austral**.

0.3 Instalar las dependencias y levantar la app

En GitHub Desktop: **Repository** → **Open in Command Prompt** (o abre la carpeta en VS Code y usa **Terminal** → **New Terminal**). Luego:

```
cd frontend
npm install      # descarga las librerías: React, Tailwind, Leaflet, etc. (1-3 minutos)
npm run dev      # levanta el servidor de desarrollo
```

Verás algo así:

```
VITE v8.1.5  ready in 386 ms
→ Local:   http://localhost:5173/
```

Abre esa dirección en Chrome. Debes ver el catálogo con 34 inmuebles.

⚠ Importante: todos los comandos **npm** se ejecutan **dentro de frontend/**, no en la raíz del repositorio. Si ves **npm error code ENOENT ... package.json**, es que estás en la carpeta equivocada.

0.4 Tres cosas que debes saber de esta app antes de tocarla

1. **No hay backend todavía.** Los datos viven en archivos JavaScript (`frontend/src/fixtures/`) y se cargan a una "base de datos falsa" en memoria (`frontend/src/api/store.js`). En la fase 2 esto se reemplaza por FastAPI + PostgreSQL.
 2. **El navegador guarda el estado.** Lo que creas en la demo (reservas, confirmaciones) queda en el `localStorage` de Chrome bajo la clave `coipo_demo_v1`. Si cambias un *fixture* y no ves el cambio, es porque el navegador está usando lo guardado: usa el botón **"Reiniciar demostración"** del pie de página.
 3. **Guardar = recargar.** Vite recarga la página sola al guardar un archivo (*hot reload*). No hay que compilar a mano.
-

Mapa del proyecto (qué hace cada carpeta)

```

coipo_cabania/
├ docs/           Documentación funcional (alcance, guion de demo, insumos)
├ INSUMO/         Requisitos originales del Servicio de Bienestar (fuente de verdad)
├ frontend/       ← acá se programa
│   ├── index.html  Página raíz. Casi nunca se toca.
│   ├── package.json Librerías y comandos (npm run dev / lint / build)
│   ├── qa/         Pruebas automatizadas con navegador real (Puppeteer)
│   └── src/
│       ├── main.jsx Punto de entrada: monta React y el enrutador
│       ├── App.jsx   Mapa de rutas: qué URL muestra qué página
│       ├── index.css Colores institucionales y estilos globales
│       ├── pages/    Una carpeta por perfil de usuario; cada archivo es una pantalla
│       │   ├── publico/ Afiliado y no afiliado (catálogo, ficha, reservar, mis reservas)
│       │   ├── regional/ Encargada regional
│       │   ├── central/ Oficina Central
│       │   └── admin/   Administrador
│       ├── components/ Piezas reutilizables (tarjetas, botones, tablas, mapas)
│       ├── api/        Capa que IMITA al backend. Toda pantalla pide los datos acá.
│       ├── fixtures/   Los datos de mentira (inmuebles, reservas, tarifas, usuarios)
│       ├── lib/         Utilidades puras (formato de fechas y pesos, cálculo de tarifas)
│       └── context/     Estado global (el rol con el que se navega)
└ .github/workflows/  Automatización: publica la maqueta en GitHub Pages
    
```

La regla de arquitectura más importante del proyecto, escrita en [client.js](#):

Ningún componente debe importar *fixtures* ni tocar *localStorage* directamente: todo pasa por la capa *api*.

¿Por qué? Porque en la fase 2 solo se reescribe *api* con llamadas *fetch* reales y **ninguna pantalla cambia**.

Flujo de datos, de arriba abajo:

```

fixtures/*.js → api/store.js → api/inmuebles.js → pages/Catalogo.jsx → components/InmuebleCard.jsx
(los datos)   (la "BD" falsa) (el "endpoint")   (la pantalla)       (la pieza visual)
    
```

Parte 1 — 15 ejercicios guiados

Cómo trabajar esta parte

- Haz los ejercicios **en orden**: cada uno usa lo aprendido en el anterior.
- Todos se hacen sobre **una misma rama de práctica** que crearás en el ejercicio 1.
- **Nada de esto se sube a main**. Al final (ejercicio 15) se descarta todo.
- Deja `npm run dev` corriendo en una terminal durante toda la sesión.

Formato de cada ejercicio:

Objetivo · Concepto · Dónde · Está así → Debe quedar así · Verificación · Si falla

Ejercicio 1 — Crear una rama en GitHub Desktop

Objetivo: trabajar sin romper lo que está publicado.

Concepto — ¿qué es una rama? El repositorio es una línea de tiempo de versiones. `main` es la línea oficial: lo que está ahí se publica automáticamente. Una **rama** (*branch*) es una copia paralela donde puedes experimentar; si rompes algo, `main` sigue intacto. Cuando el trabajo está listo, se pide fusionarlo a `main` mediante un **Pull Request** (parte 2).

Pasos

1. Abre **GitHub Desktop**. Arriba verás tres cajas: `Current repository`, `Current branch`, y `Fetch origin`.
2. Confirma que `Current repository` diga `coipo_cabania`.
3. Haz clic en `Fetch origin` (trae los cambios que otros subieron).
4. Clic en `Current branch` → asegúrate de estar en `main` → si no, selecciónala.
5. Clic en `Current branch` otra vez → botón azul **New branch**.
6. Nombre: `practica/tu-nombre` (ejemplo: `practica/juan-perez`).
 - Sin espacios, sin tildes, sin mayúsculas. Usa `/` y `-`.
 - Verifica que abajo diga `"based on main"`.
7. Clic en **Create branch**.
8. Aparece un botón **Publish branch**: haz clic (sube la rama a GitHub para que exista también en el servidor).

Verificación `Current branch` ahora dice `practica/tu-nombre`. En [GitHub.com](https://github.com) → pestaña `branches` aparece tu rama.

Si falla

- `"Your branch has unpublished changes"`: es normal antes de publicar; publica la rama.
- Si **New branch** está deshabilitado, tienes cambios sin guardar: guarda los archivos en VS Code primero.

Convención de nombres del proyecto

Prefijo	Cuándo	Ejemplo
<code>feat/</code>	funcionalidad nueva	<code>feat/valoraciones-estrellas</code>
<code>fix/</code>	corrección de un error	<code>fix/desborde-tabla-movil</code>
<code>docs/</code>	solo documentación	<code>docs/guia-induccion</code>
<code>practica/</code>	pruebas de aprendizaje (no se fusionan)	<code>practica/juan-perez</code>

Ejercicio 2 — Cambiar un texto en pantalla

Objetivo: entender qué es un componente y ver el ciclo *editar* → *guardar* → *ver*.

Concepto — componente y JSX Cada pantalla es una **función de JavaScript** que devuelve algo parecido a HTML. Ese "HTML dentro de JavaScript" se llama **JSX**. La función `Catalogo()` devuelve la página del catálogo; el navegador no ejecuta JSX directamente: Vite lo traduce a JavaScript.

Dónde: `frontend/src/pages/publico/Catalogo.jsx`, línea 47.

Está así

```
<TituloSeccion
  titulo="Catálogo de inmuebles"
  descripcion="Red de 34 casas de huéspedes y de veraneo del Servicio de Bienestar..."
/>
```

Debe quedar así

```
<TituloSeccion
  titulo="Catálogo de inmuebles – versión de práctica"
  descripcion="Red de 34 casas de huéspedes y de veraneo del Servicio de Bienestar..."
/>
```

Verificación Guarda (**Ctrl+S**). El navegador se actualiza solo en menos de un segundo y el título grande cambió. **No recargues a mano**: si tienes que recargar, el *hot reload* falló.

Si falla

- Nada cambia → ¿guardaste? ¿estás mirando <http://localhost:5173/...> y no el sitio publicado en GitHub Pages?
- Pantalla en blanco → seguro borraste una comilla. Abre la consola de Chrome (**F12** → pestaña **Console**) y lee el mensaje rojo: dirá el archivo y la línea.

Concepto extra — cómo leer `<TituloSeccion titulo="..." />` `TituloSeccion` es un componente definido en [components/ui/Elementos.jsx](#). Lo que va dentro de la etiqueta (`titulo=`, `descripcion=`) son **props**: los parámetros del componente. Es exactamente como llamar a una función con argumentos con nombre.

Ejercicio 3 — Cambiar estilos con Tailwind

Objetivo: entender cómo se aplican estilos en este proyecto.

Concepto — Tailwind CSS No hay archivos `.css` por componente. El estilo se escribe como **clases utilitarias** en el atributo `className`: `p-4` = padding, `mt-2` = margen superior, `text-sm` = letra chica, `bg-verde-600` = fondo verde institucional. Los colores `verde-*` y `arena-*` son propios de CONAF y están definidos en [index.css](#).

Dónde: [frontend/src/components/inmuebles/InmubleCard.jsx](#), línea **43**.

Está así

```
<h3 className="text-base leading-snug font-semibold text-verde-900 group-hover:underline">
```

Debe quedar así

```
<h3 className="text-lg leading-snug font-bold text-rose-700 group-hover:underline">
```

Verificación Los nombres de los 34 inmuebles del catálogo se ven más grandes, en negrita y rojos.

Si falla

- El color no cambia → Tailwind solo genera las clases que encuentra escritas completas. `text-rose-700` funciona; `text-rose-` + variable **no**.

- Clase inventada (`text-rojo-fuerte`) → no pasa nada, no da error. Consulta los nombres válidos en <https://tailwindcss.com/docs> o pasa el mouse sobre la clase en VS Code con la extensión Tailwind IntelliSense.

Cuando termines: devuelve la línea a su estado original (`text-base`, `font-semibold`, `text-verde-900`). Los ejercicios siguientes asumen el diseño normal.

Ejercicio 4 — Cambiar un dato y entender de dónde viene

Objetivo: distinguir **datos** de **presentación**, y descubrir el `localStorage`.

Concepto — fixtures Los 34 inmuebles son un arreglo de objetos JavaScript en `fixtures/inmuebles.js`. Cada objeto tiene la forma que tendrá mañana una fila de la base de datos (por eso los nombres van en `snake_case`: `capacidad_maxima`, no `capacidadMaxima`).

Dónde: `frontend/src/fixtures/inmuebles.js`, inmueble `id: 1`, líneas **43** y **50**.

Está así

```
{
  id: 1,
  nombre: 'Casa de Huéspedes Toconao',
  ...
  capacidad_maxima: 6,
```

Debe quedar así

```
{
  id: 1,
  nombre: 'Casa de Huéspedes Toconao (en mantención)',
  ...
  capacidad_maxima: 12,
```

Verificación

1. Ve al catálogo y busca "Toconao" en el buscador.
2. **Si ya habías usado la demo** (creaste o confirmaste una reserva alguna vez en este navegador), **NO verás el cambio**. Ese es el aprendizaje del ejercicio. Si es tu primera vez, el cambio se ve de inmediato: provócalo entonces creando primero una reserva cualquiera y repitiendo el ejercicio.
3. Baja al pie de página → **"Reiniciar demostración"** → `Reiniciar`.
4. Ahora sí: el nombre cambió y dice "Hasta 12 personas".

Por qué pasa `api/store.js` primero busca en `localStorage`; si hay algo guardado, ignora los *fixtures*. "Reiniciar demostración" borra esa clave.

Si falla

- `Uncaught SyntaxError: Unexpected token` → te faltó una coma entre propiedades del objeto, o borraste una comilla.
- Sigue sin cambiar tras reiniciar → `F12` → pestaña `Application` → `Local Storage` → borra la clave `coipo_demo_v1` a mano y recarga.

Cuando termines: revierte el nombre y la capacidad a los valores originales ('Casa de Huéspedes Toconao', 6).

Ejercicio 5 — Props: pasarle información a un componente

Objetivo: entender cómo un componente padre le manda datos a un hijo.

Concepto — props `InmuableCard` no sabe nada del catálogo: recibe un inmueble y lo dibuja. Fíjate en la firma, línea **18**:

```
export function InmuableCard({ inmueble, esExterno = false }) {
```

Recibe dos props; `esExterno` tiene **valor por defecto** `false`, así que es opcional. Quien la usa es `Catalogo.jsx` línea **153**:

```
<InmuableCard inmueble={inmueble} esExterno={esNoAfiliado} />
```

Ejercicio: agrega una prop `destacado` que dibuje un borde verde grueso.

Cambio 1 — `InmuableCard.jsx` línea **18**:

```
export function InmuableCard({ inmueble, esExterno = false, destacado = false }) {
```

Cambio 2 — mismo archivo, línea **24** (el `className` del `<Link>`):

```
className={`group flex flex-col overflow-hidden rounded-xl border bg-white shadow-sm transition-shadow
  destacado ? 'border-4 border-verde-600' : 'border-arena-200'
}`}
```

Nota: las comillas invertidas ``` (*template literal*) permiten meter una expresión con `${...}` dentro del texto. Es JavaScript puro, no algo de React.

Cambio 3 — `Catalogo.jsx` línea **153**:

```
<InmuableCard
  inmueble={inmueble}
  esExterno={esNoAfiliado}
  destacado={inmueble.tipo === 'veraneo'}
/>
```

Verificación Las casas de **veraneo** (badge verde) quedan con borde grueso; las de huéspedes no.

Si falla

- Todas con borde → `inmueble.tipo === 'veraneo'` con **tres iguales**; con uno solo (`=`) estarías asignando, no comparando.
- Ninguna con borde → revisa que en el `className` estés usando comillas invertidas y no comillas dobles.

Cuando termines: revierte los tres cambios (`Ctrl+Z` sirve; o descarta el archivo en GitHub Desktop con clic derecho → `Discard changes`).

Ejercicio 6 — Estado con `useState`: hacer que la interfaz reaccione

Objetivo: entender por qué una variable normal no basta para cambiar la pantalla.

Concepto — estado Una variable común cambia en memoria pero React no se entera y no vuelve a dibujar. El **estado** es una variable que, al cambiar, obliga al componente a redibujarse:

```
const [abierto, setAbierto] = useState(false)
//      ↑ valor    ↑ función para cambiarlo  ↑ valor inicial
```

Dónde: `frontend/src/pages/publico/FichaInmueble.jsx`.

Cambio 1 — dentro de la función `FichaInmueble()`, junto a los otros `useState` (líneas **90-91**), agrega:

```
const [verCondiciones, setVerCondiciones] = useState(true)
```

Cambio 2 — línea **198**, reemplaza el encabezado "Condiciones de uso":

Está así:

```
<h3 className="mt-5 text-sm font-semibold text-slate-800">Condiciones de uso</h3>
<ul className="mt-2 list-disc space-y-1 pl-5 text-sm text-slate-600">
```

Debe quedar así:

```
<button
  type="button"
  onClick={() => setVerCondiciones(!verCondiciones)}
  className="mt-5 cursor-pointer text-sm font-semibold text-verde-700 underline"
>
  {verCondiciones ? 'Ocultar' : 'Ver'} condiciones de uso
</button>
<ul
  className={`mt-2 list-disc space-y-1 pl-5 text-sm text-slate-600 ${
    verCondiciones ? '' : 'hidden'
  }`}
>
```

Verificación Entra a cualquier ficha (clic en una tarjeta del catálogo). El texto "Ocultar condiciones de uso" es ahora un botón: al hacer clic la lista desaparece y el texto cambia a "Ver condiciones de uso".

Si falla

- `useState is not defined` → falta importarlo. La línea 1 del archivo ya dice `import { useEffect, useState } from 'react'`; si la modificaste, restáurala.
- El clic no hace nada → `onClick={setVerCondiciones(!verCondiciones)}` (sin la flecha) ejecuta la función al dibujar, no al hacer clic. Debe ser `onClick={() => setVerCondiciones(!verCondiciones)}`.
- `"Too many re-renders"` → mismo error anterior: cambiaste el estado durante el dibujo y React entró en bucle.

Cuando termines: descarta los cambios del archivo.

Ejercicio 7 — Renderizado condicional

Objetivo: mostrar cosas solo cuando se cumple una condición.

Concepto — tres formas, todas en el código del proyecto

Forma	Se lee	Ejemplo real
<code>{cond && <X />}</code>	"si <code>cond</code> , muestra <code>X</code> "	FichaInmueble.jsx:142 <code>{esPortal && <Boton...>}</code>
<code>{cond ? <A /> : }</code>	"si <code>cond</code> muestra A, si no B"	InmuebleCard.jsx:36 tono del badge
<code>if (...) return ...</code>	corta antes de dibujar	FichaInmueble.jsx:105 <code>if (cargando) return <Cargando /></code>

Ejercicio: mostrar un aviso solo en inmuebles grandes.

Dónde: [FichaInmueble.jsx](#), justo antes de la `<Tarjeta className="p-5">` de "Descripción" (línea **154**), agrega:

```
{inmueble.capacidad_maxima >= 8 && (
  <Aviso tono="verde" titulo="Inmueble de alta capacidad">
    Este inmueble admite {inmueble.capacidad_maxima} personas: sirve para
    comisiones de servicio numerosas.
  </Aviso>
)}
```

Verificación

- Ficha del inmueble **id 1** (Toconao, capacidad 6): **no** aparece el aviso.
- Busca en el catálogo un inmueble que diga "Hasta 8 personas" o más: **sí** aparece.

Si falla

- `Aviso is not defined` → ya está importado en la línea 22 del archivo; si lo borraste, restaura `import { Aviso, Boton, Cargando, Tarjeta } from '../components/ui/Elementos.jsx'`.
- Aparece un `0` suelto en pantalla → clásico de React: si escribes `{inmueble.dormitorios && <X/>}` y `dormitorios` vale `0`, React dibuja el `0`. Por eso se usa una comparación (`>= 8`, `> 0`) y no el número pelado.

Cuando termines: descarta el cambio.

Ejercicio 8 — Listas y la prop `key`

Objetivo: entender cómo se dibuja una lista a partir de un arreglo.

Concepto — `.map()` `arreglo.map(f)` transforma cada elemento. En JSX se usa para convertir datos en elementos visuales. React exige una prop `key` única por elemento para saber cuál es cuál cuando la lista cambia.

Dónde: [Catalogo.jsx](#), líneas **97-101**:

```
{[2, 4, 5, 6, 8, 10].map((n) => (
  <option key={n} value={n}>
    {n} personas o más
  </option>
))}
```

Cambio: agrega dos opciones al arreglo:

```
{[2, 4, 5, 6, 8, 10, 12, 15].map((n) => (
```

Verificación El selector "Capacidad mínima" ahora tiene "12 personas o más" y "15 personas o más". Elige 12: el catálogo dirá "0 de 34 inmuebles" y aparecerá el estado vacío (ningún inmueble de la maqueta tiene esa capacidad). Elige 8: quedan solo los grandes.

Experimento obligatorio: borra `key={n}` y guarda. Abre `F12` → `Console`. Verás:

```
Warning: Each child in a list should have a unique "key" prop.
```

La app sigue funcionando, pero es un *warning* que nunca debe quedar en el código. Restaura la `key`.

Si falla

- El filtro no filtra nada → el filtro se aplica en `api/inmuebles.js:20` (`i.capacidad_maxima >= Number(capacidad_min)`); si lo tocaste, restáuralo.

Cuando termines: deja el arreglo original `[2, 4, 5, 6, 8, 10]`.

Ejercicio 9 — Eventos y formularios controlados

Objetivo: agregar un filtro nuevo de punta a punta (interfaz + capa API).

Concepto — componente controlado Un `<input>` en React no guarda su propio valor: lo lee del estado (`value={...}`) y avisa los cambios (`onChange={...}`). El estado manda; el input solo refleja. En `Catalogo.jsx` todos los filtros viven en un único objeto de estado (línea **22**):

```
const [filtros, setFiltros] = useState(FILTROS_VACIOS)
```

Ejercicio: agregar un filtro "solo inmuebles con Wi-Fi".

Cambio 1 — `Catalogo.jsx` línea **18**:

```
const FILTROS_VACIOS = { region: '', tipo: '', capacidad_min: '', busqueda: '', wifi: '' }
```

Cambio 2 — `Catalogo.jsx`, después del `</Campo>` del buscador (línea **120**), agrega un cuarto campo:

```

<Campo etiqueta="Conectividad">
  <label className="flex min-h-11 items-center gap-2 text-sm text-slate-700">
    <input
      type="checkbox"
      checked={filtros.wifi === 'si'}
      onChange={(e) =>
        setFiltros((f) => ({ ...f, wifi: e.target.checked ? 'si' : '' })))
    />
    Solo inmuebles con Wi-Fi
  </label>
</Campo>

```

Cambio 3 — `api/inmuebles.js`, línea **14** (agregar `wifi` a lo que se desestructura) y línea **20** (el filtro nuevo):

Está así:

```
const { region, tipo, capacidad_min, localidad, busqueda } = filtros
```

Debe quedar así:

```
const { region, tipo, capacidad_min, localidad, busqueda, wifi } = filtros
```

Y después de la línea del `capacidad_min`, agrega:

```
if (wifi === 'si') items = items.filter((i) => i.equipamiento.includes('Wi-Fi'))
```

Verificación Marca la casilla: el contador baja (los inmuebles de montaña no tienen Wi-Fi en la maqueta). Desmárcala: vuelve a "34 de 34". El botón "Limpiar filtros" también la desmarca, porque lee de `FILTROS_VACIOS`.

Si falla

- El contador no cambia → revisa el `useEffect` de la línea 26: depende de `[filtros]`, así que cualquier cambio del objeto vuelve a pedir los datos. Si no se dispara, es que mutaste el objeto en vez de crear uno nuevo. **En React el estado nunca se muta:** se reemplaza con `{ ...f, wifi: ... }`.
- "Limpiar filtros" no aparece → `hayFiltros` (línea 37) revisa que algún valor sea distinto de `''`; por eso el valor del checkbox es `'si' / ''` y no `true / false`.

Cuando termines: descarta los dos archivos.

Ejercicio 10 — La capa API y el código asíncrono

Objetivo: entender por qué las pantallas no leen los *fixtures* directamente.

Concepto — **promesas** y `async` `api/client.js` simula la demora de una red:

```
const LATENCIA_MS = 180

export function responder(datos, ms = LATENCIA_MS) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(estructurar(datos)), ms)
  })
}
```

Una **promesa** es un valor que llegará después. Por eso las pantallas escriben `getInmuebles(...).then((r) => setInmuebles(r.items))`: "cuando llegue, guárdalo en el estado". Gracias a esto, cuando exista el backend real solo cambia `api/`.

Ejercicio: exagerar la latencia para ver los estados de carga.

Dónde: `api/client.js` línea **14**.

Está así

```
const LATENCIA_MS = 180
```

Debe quedar así

```
const LATENCIA_MS = 3000
```

Verificación Recarga el catálogo: durante 3 segundos verás "Buscando..." y el spinner "Cargando inmuebles...". Esos estados existen en `Catalogo.jsx:136-137` y son obligatorios: toda pantalla que pide datos debe mostrar algo mientras espera.

Extra: entra a la ficha de un inmueble y presiona rápido "Volver al catálogo". No pasa nada raro, porque el `useEffect` cancela la respuesta atrasada con la bandera `vigente` (`Catalogo.jsx:26-35`). Sin esa bandera, una respuesta vieja podría sobrescribir datos nuevos.

Cuando termines: vuelve a **180**. **Nunca** subas este archivo con otro valor.

Ejercicio 11 — `useEffect`: efectos y limpieza

Objetivo: entender cuándo se ejecuta el código que pide datos.

Concepto `useEffect(fn, [deps])` ejecuta `fn` después de dibujar, y lo repite cada vez que cambia algo del arreglo `[deps]`. Lo que la función `fn` **devuelve** es la limpieza: se ejecuta antes de repetir o al desmontar el componente.

Ejercicio: instrumentar el efecto del catálogo.

Dónde: `Catalogo.jsx` línea **26**.

Debe quedar así (se agregan las dos líneas con `console.log`):

```
useEffect(() => {
  let vigente = true
  console.log('EFECTO: pidiendo inmuebles con', filtros)
  setCargando(true)
  getInmuebles(filtros)
    .then((r) => vigente && setInmuebles(r.items))
    .finally(() => vigente && setCargando(false))
  return () => {
    console.log('LIMPIEZA: se descarta la petición anterior')
    vigente = false
  }
}, [filtros])
```

Verificación (F12 → Console)

1. Al entrar al catálogo verás el mensaje **dos veces**. No es un error: en desarrollo React usa **StrictMode** ([main.jsx:20](#)) y monta cada componente dos veces para detectar efectos mal escritos. En producción ocurre una vez.
2. Escribe en el buscador letra por letra: cada tecla dispara "LIMPIEZA" y luego "EFECTO". Así se ve que el arreglo de dependencias manda.

Experimento: cambia `}, [filtros])` por `})` (sin arreglo) y mira la consola. Bucle infinito: el efecto cambia el estado, el estado redibuja, el redibujado dispara el efecto... **Restáuralo de inmediato** (**Ctrl+Z**); deja la pestaña sin recargar mientras tanto.

Si falla

- No ves nada en consola → asegúrate de estar en la pestaña **Console** y con el filtro en **All levels**.

Cuando termines: descarta el archivo.

Ejercicio 12 — Rutas: crear una página nueva

Objetivo: entender cómo una URL se convierte en pantalla.

Concepto — enrutador **App.jsx** es el mapa: cada `<Route path="..." element={...} />` asocia una URL con un componente. Las URL son limpias (por ejemplo `.../catalogo`, sin `#`) porque la aplicación usa **BrowserRouter** ([main.jsx:8-18](#)). Para que recargar una ruta profunda no dé 404, cada destino hace su parte: en el servidor CONAF el nginx del contenedor devuelve `index.html` para cualquier ruta que no sea un archivo, y en GitHub Pages ese papel lo cumple el `404.html` que copia el workflow de publicación.

Cambio 1 — crea el archivo `frontend/src/pages/publico/Ayuda.jsx`:

```
import { TituloSeccion, Tarjeta } from '../../components/ui/Elementos.jsx'

export function Ayuda() {
  return (
    <>
      <TituloSeccion
        titulo="Ayuda"
        descripcion="Preguntas frecuentes sobre el uso del sistema de reservas."
      />
      <Tarjeta className="p-5">
        <h2 className="text-lg font-semibold text-verde-900">¿Cómo reservo?</h2>
        <p className="mt-2 text-sm text-slate-700">
          Busque el inmueble en el catálogo, revise la disponibilidad en su ficha y
          presione «Solicitar reserva».
        </p>
      </Tarjeta>
    </>
  )
}
```

Cambio 2 — [App.jsx](#) línea 8, agrega el import:

```
import { Ayuda } from './pages/publico/Ayuda.jsx'
```

Cambio 3 — [App.jsx](#) línea 31, después de la ruta del catálogo:

```
<Route path="ayuda" element={ <Ayuda /> } />
```

Cambio 4 — [components/layout/AppLayout.jsx](#) línea 30, agrega el enlace al menú del afiliado:

```
afiliado: [
  { a: '/catalogo', texto: 'Catálogo', icono: LayoutGrid },
  { a: '/mis-reservas', texto: 'Mis reservas', icono: ClipboardList },
  { a: '/ayuda', texto: 'Ayuda', icono: Info },
],
```

(Info ya está importado en la línea 10 del archivo.)

Verificación

1. Aparece "Ayuda" en la barra verde (solo con el perfil Afiliado/a).
2. Al hacer clic, la URL queda `.../ayuda` y se ve la página nueva.
3. Escribe a mano `.../ayudaa` (mal escrito): te devuelve al catálogo, por la ruta comodín `<Route path="*" ...>` de la línea 137.
4. Cambia el rol a "Encargada regional": el enlace desaparece, porque cada perfil tiene su propio menú.

Si falla

- `Ayuda is not defined` → falta el import (cambio 2) o el nombre no coincide. El nombre exportado (`export function Ayuda`) debe ser idéntico al importado, entre llaves.
- Página en blanco al entrar → revisa que el archivo empiece con el `import` y que el componente devuelva JSX.
- El enlace aparece pero no navega → el `a:` del menú debe ser exactamente `/ayuda`, con la barra inicial.

Cuando termines: descarta los 3 archivos y **elimina** `Ayuda.jsx` (en GitHub Desktop aparece como archivo nuevo; clic derecho → `Discard changes` también lo borra).

Ejercicio 13 — Contexto: el rol activo

Objetivo: entender cómo un dato global llega a cualquier pantalla sin pasarlo por props en cada nivel.

Concepto — Context El selector de perfil del encabezado guarda el rol en un **contexto** (`context/RolContext.jsx`). Cualquier componente lo lee con `useRol()`:

```
const { rol, usuario, esAfiliado, esNoAfiliado, esPortal, actor } = useRol()
```

Además, `RequiereRol.jsx` protege las rutas: si el perfil no corresponde, no muestra la pantalla. En la fase 2 esto se reemplaza por la sesión real; el resto del código no cambia.

Ejercicio: mostrar un saludo personalizado solo a los afiliados.

Dónde: `Catalogo.jsx` línea 21.

Está así

```
const { esNoAfiliado } = useRol()
```

Debe quedar así

```
const { esNoAfiliado, esAfiliado, usuario } = useRol()
```

Y después del `<TituloSeccion .../>` (línea 49), agrega:

```
{esAfiliado && (
  <p className="mb-4 text-sm text-slate-600">
    Hola, <strong className="text-verde-800">{usuario.nombre}</strong>. Su unidad es{' '}
    {usuario.unidad}.
  </p>
)}
```

Verificación Con perfil **Afiliado/a** aparece "Hola, María Fuentes Rojas...". Cambia a **Usuario no afiliado**: desaparece. Cambia a **Administrador**: tampoco aparece, y además el menú cambia por completo.

Si falla

- `Cannot read properties of undefined (reading 'nombre')` → hay perfiles sin usuario asociado. Por eso el resto del código escribe `usuario?.nombre` con **interrogación** (*optional chaining*): si `usuario` es `undefined`, devuelve `undefined` en vez de reventar. Corrígelo así y compara.

- `useRol` debe usarse dentro de `RolProvider` → estás llamando `useRol()` fuera del árbol de la app (por ejemplo en un archivo de `lib/`). Los *hooks* solo se usan dentro de componentes.

Cuando termines: descarta el archivo.

Ejercicio 14 — Romper algo a propósito y aprender a leer el error

Objetivo: no tenerle miedo a la pantalla en blanco.

Haz **uno a la vez**, mira el resultado, y deshaz con **Ctrl+Z** antes del siguiente.

#	Rompe esto	En	Qué verás	Qué significa
A	Borra la <code>/</code> de cierre en <code><MapPin size={14} ...></code>	<code>InmuableCard.jsx:48</code>	Error rojo en el navegador: <i>Unterminated JSX contents</i>	En JSX toda etiqueta se cierra: <code><X /></code> o <code><X></X></code>
B	Cambia <code>inmuable.nombre</code> por <code>inmuable.Nombre</code>	<code>InmuableCard.jsx:44</code>	Las tarjetas quedan sin título (no hay error)	JavaScript distingue mayúsculas; una propiedad inexistente es <code>undefined</code> y React no dibuja nada
C	Cambia <code>getInmuebles(filtros)</code> por <code>getInmuable(filtros)</code>	<code>Catalogo.jsx:29</code>	<code>getInmuable is not defined</code>	No está importado: los imports son explícitos, arriba del archivo
D	Borra <code>esExterno={esNoAfiliado}</code>	<code>Catalogo.jsx:153</code>	Las tarifas "Desde" cambian para el perfil no afiliado	La prop tomó su valor por defecto <code>false</code>
E	Escribe <code>{inmuable.servicios.length}</code> en la tarjeta	<code>InmuableCard.jsx:45</code>	Pantalla en blanco: <code>Cannot read properties of undefined (reading 'length')</code>	Ese campo no existe en los datos. Se resuelve con <code>inmuable.servicios?.length ?? 0</code> , que devuelve <code>0</code> en vez de reventar

Cómo leer un error de React en Chrome

1. La pantalla roja de Vite muestra **archivo:línea** arriba de todo. Empieza por ahí.
2. **F12** → **Console**. Ignora el ruido: busca la **primera** línea roja, y dentro de ella el nombre de un archivo del proyecto (`src/...`), no de una librería.
3. **F12** → **Components** (si instalaste React DevTools) permite inspeccionar props y estado en vivo.
4. Si la app quedó rara sin razón aparente: pie de página → "Reiniciar demostración".

Verificación: pudiste provocar y explicar los cinco errores. Es el ejercicio más importante de la parte 1: **el 80 % del trabajo real es leer errores**.

Ejercicio 15 — Cerrar la práctica: revisar, verificar y descartar

Objetivo: dejar el repositorio limpio y conocer los controles de calidad.

Paso 1 — revisa qué tocaste En GitHub Desktop, pestaña **Changes**: aparecen los archivos modificados. Verde = líneas agregadas, rojo = eliminadas. **Léelas todas:** es exactamente lo que hace un revisor en un Pull Request.

Paso 2 — corre los dos controles de calidad

```
cd frontend
npm run lint    # revisa reglas de código (hooks mal usados, etc.)
npm run build   # compila la versión de producción
```

`npm run lint` debe terminar mostrando, como máximo, estos **3 warnings preexistentes**:

```
src/components/charts/graficos.jsx:29:14  warning  react(only-export-components)
src/context/RolContext.jsx:54:17           warning  react(only-export-components)
src/components/layout/AppLayout.jsx:53:14  warning  react(only-export-components)
```

Si aparece uno nuevo o un **error**, es tuyo: corrígelo. `npm run build` debe terminar con **✓ built in ...s**. El aviso sobre *chunks* mayores a 500 kB también es preexistente.

Paso 3 — descarta todo En **Changes**: clic derecho sobre la lista → **Discard all changes...** → confirma. Los archivos vuelven al estado de **main**. (Esto es irreversible; por eso se hace solo en ramas de práctica.)

Paso 4 — vuelve a **main** y borra la rama

1. **Current branch** → **main**.
2. **Current branch** → clic derecho sobre **practica/tu-nombre** → **Delete...**
3. Marca también "borrar en el remoto" si lo ofrece.

Qué pasa cuando el código llega a **main** [.github/workflows/pages.yml](#) ejecuta automáticamente `npm ci`, `npm run lint` y `npm run build`, y publica la maqueta en GitHub Pages. Si el lint o el build fallan, **no se publica**. Ese pipeline corre al fusionar a **main**, no durante el Pull Request: por eso ambos comandos se corren a mano antes de pedir revisión.

Autoevaluación de la parte 1

Deberías poder responder sin mirar:

1. ¿Cuál es la diferencia entre una prop y un estado?
2. ¿Por qué una pantalla nunca importa un archivo de **fixtures/** directamente?
3. ¿Qué hace el arreglo de dependencias de **useEffect**?
4. ¿Por qué a veces un cambio en un *fixture* no se refleja en la pantalla?
5. ¿Dónde se define qué URL muestra qué pantalla?
6. ¿Qué dos comandos hay que correr antes de pedir revisión?

Parte 2 — Funcionalidad completa: valoración con 5 estrellas

Ahora se construye una funcionalidad real, completa, con la misma secuencia que se usa en el trabajo diario: rama → código → pruebas → commits → Pull Request.

Nota: esta implementación fue construida y verificada en el navegador antes de escribir esta guía. Los promedios, textos y comportamientos que se indican en las verificaciones son los observados realmente.

2.0 Qué se va a construir

Historia de usuario

Como afiliado que ya se alojó en una casa de la red, quiero valorar mi estadía con 1 a 5 estrellas y un comentario, para que otros funcionarios sepan cómo es el inmueble antes de reservar.

Reglas de negocio

1. Solo puede valorar quien tuvo una reserva en estado **finalizada**.
2. Una valoración por reserva (no se puede valorar dos veces la misma estadía).
3. La nota es un entero de 1 a 5; el comentario es opcional (máx. 300 caracteres).
4. En el catálogo y en la ficha se muestra el **promedio** y la **cantidad** de valoraciones.
5. Toda valoración queda en la pista de auditoría (Ley 19.628 / 21.719).

Qué se ve al final

- **Catálogo:** cada tarjeta con valoraciones muestra ★★★★★ 4.5 (2) bajo el nombre.
- **Ficha del inmueble:** el promedio bajo el título y una tarjeta nueva "Valoraciones de huéspedes" con los comentarios.
- **Mis reservas:** las reservas finalizadas muestran un botón **Valorar**; al valorar, el botón se reemplaza por las estrellas de la nota dada.

Archivos que se tocan (3 nuevos, 5 modificados)

#	Archivo	Acción	Por qué
1	<code>frontend/src/fixtures/valoraciones.seed.js</code>	nuevo	Datos de ejemplo
2	<code>frontend/src/api/store.js</code>	modificar	Registrar la colección nueva en la "BD"
3	<code>frontend/src/api/valoraciones.js</code>	nuevo	Endpoints (GET y POST) y reglas
4	<code>frontend/src/components/inmuebles/Estrellas.jsx</code>	nuevo	Las estrellas (ver y elegir)
5	<code>frontend/src/api/inmuebles.js</code>	modificar	Adjuntar el promedio a cada inmueble
6	<code>frontend/src/components/inmuebles/InmuebleCard.jsx</code>	modificar	Mostrar el promedio en el catálogo
7	<code>frontend/src/pages/publico/FichaInmueble.jsx</code>	modificar	Promedio + lista de comentarios
8	<code>frontend/src/api/reservas.js</code>	modificar	Saber si la reserva ya fue valorada
9	<code>frontend/src/pages/publico/MisReservas.jsx</code>	modificar	Botón y formulario para valorar

Orden de trabajo (de abajo hacia arriba): primero los datos, después la lógica, después la interfaz. Así cada paso se puede probar apoyado en el anterior.

Sobre los números de línea: corresponden al archivo **tal como está en main hoy**. Al agregar líneas, las siguientes se desplazan. Por eso cada paso incluye el **texto de referencia**: búscalo con **Ctrl+F** en VS Code en lugar de confiar solo en el número.

2.1 Paso 0 — Crear la rama

1. GitHub Desktop → **Fetch origin**.

2. **Current branch** → **main** (importante: la rama se crea desde **main** actualizado).
3. **Current branch** → **New branch** → nombre: **feat/valoraciones-estrellas** → **Create branch**.
4. **Publish branch**.

Deja corriendo en una terminal:

```
cd frontend  
npm run dev
```

2.2 Paso 1 — Los datos de ejemplo (archivo nuevo)

Crea `frontend/src/fixtures/valoraciones.seed.js`.

En VS Code: clic derecho sobre la carpeta `src/fixtures` → **New File...** → escribe el nombre exacto (respeta mayúsculas y el `.seed.js`).

```
/**
 * Valoraciones de ejemplo – DATOS FICTICIOS.
 *
 * Cada valoración pertenece a una reserva finalizada de `reservas.seed.js`:
 * solo quien se alojó puede valorar. Las estrellas van de 1 a 5.
 */

export const valoracionesSeed = [
  {
    id: 1,
    inmueble_id: 9,
    reserva_codigo: 'R-2026-0006',
    usuario_id: 13,
    autor: 'Pedro Millán Quezada',
    estrellas: 5,
    comentario: 'Muy buena ubicación para el cometido y la casa estaba impecable.',
    fecha: '2026-07-24T12:10:00',
  },
  {
    id: 2,
    inmueble_id: 9,
    reserva_codigo: 'R-2026-0010',
    usuario_id: 12,
    autor: 'Ana Villalobos Díaz',
    estrellas: 4,
    comentario: 'Cómoda y bien equipada. La calefacción cuesta un poco al principio.',
    fecha: '2026-06-18T11:00:00',
  },
  {
    id: 3,
    inmueble_id: 7,
    reserva_codigo: 'R-2026-0007',
    usuario_id: 11,
    autor: 'Luis Cárdenas Soto',
    estrellas: 5,
    comentario: 'Excelente vista y muy tranquilo. Volveríamos sin dudarlo.',
    fecha: '2026-07-16T12:30:00',
  },
  {
    id: 4,
    inmueble_id: 4,
    reserva_codigo: 'R-2026-0015',
    usuario_id: 16,
    autor: 'Javiera Toro Sepúlveda',
    estrellas: 3,
    comentario: 'Cumple para una estadía corta, pero faltaba loza en la cocina.',
    fecha: '2026-07-09T10:20:00',
  },
  {
```

```
id: 5,
inmueble_id: 14,
reserva_codigo: 'R-2026-0017',
usuario_id: 18,
autor: 'Ximena Bravo Alarcón',
estrellas: 4,
comentario: 'Buena atención de la encargada y entrega puntual de llaves.',
fecha: '2026-06-25T09:45:00',
},
{
id: 6,
inmueble_id: 25,
reserva_codigo: 'R-2026-0019',
usuario_id: 14,
autor: 'Soledad Ríos Peña',
estrellas: 5,
comentario: 'Muy cerca del hospital, fue clave para el control médico.',
fecha: '2026-07-16T14:00:00',
},
{
id: 7,
inmueble_id: 5,
reserva_codigo: 'R-2026-0030',
usuario_id: 16,
autor: 'Javiera Toro Sepúlveda',
estrellas: 4,
comentario: 'Ideal para ir con niños. La playa queda a pasos.',
fecha: '2026-01-19T13:15:00',
},
{
id: 8,
inmueble_id: 22,
reserva_codigo: 'R-2026-0032',
usuario_id: 11,
autor: 'Luis Cárdenas Soto',
estrellas: 2,
comentario: 'La casa necesita mantención: una ducha sin agua caliente.',
fecha: '2026-02-08T10:00:00',
},
]
```

Por qué está así

- Los `reserva_codigo` existen de verdad en `reservas.seed.js` y están en estado `finalizada`. Datos inventados que no calcen romperían la coherencia de la demo.
- Los nombres de campo van en `snake_case` porque así los devolverá FastAPI en la fase 2.
- La fecha usa formato ISO (`2026-07-24T12:10:00`), igual que el resto del proyecto.

Verificación de este paso: ninguna todavía — el archivo aún no lo usa nadie. La app debe seguir funcionando exactamente igual. Si se rompió, hay un error de sintaxis: revisa comas y comillas.

2.3 Paso 2 — Registrar la colección en la "base de datos" falsa

Archivo: [frontend/src/api/store.js](#) — tres cambios.

Cambio 2.1 — importar la semilla (línea 19)

Busca: `} from '../fixtures/gestion.seed.js'`

Está así (líneas 14-19):

```
import {
  sancionesSeed,
  nominasSeed,
  auditoriaSeed,
  ultimaCargaNomina,
} from '../fixtures/gestion.seed.js'
```

Debe quedar así (se agrega **una** línea al final):

```
import {
  sancionesSeed,
  nominasSeed,
  auditoriaSeed,
  ultimaCargaNomina,
} from '../fixtures/gestion.seed.js'
import { valoracionesSeed } from '../fixtures/valoraciones.seed.js'
```

Cambio 2.2 — sumarla al estado inicial (línea 32)

Busca: `carga_nomina: ultimaCargaNomina,`

Está así (líneas 23-34):

```
function semillas() {
  return {
    inmuebles: inmueblesSeed,
    reservas: reservasSeed,
    temporadas: temporadasSeed,
    bloqueos: bloqueosTemporada,
    sanciones: sancionesSeed,
    nominas: nominasSeed,
    auditoria: auditoriaSeed,
    carga_nomina: ultimaCargaNomina,
  }
}
```

Debe quedar así:

```
function semillas() {
  return {
    inmuebles: inmueblesSeed,
    reservas: reservasSeed,
    temporadas: temporadasSeed,
    bloqueos: bloqueosTemporada,
    sanciones: sancionesSeed,
    nominas: nominasSeed,
    auditoria: auditoriaSeed,
    carga_nomina: ultimaCargaNomina,
    valoraciones: valoracionesSeed,
  }
}
```

Cambio 2.3 — que la app no se caiga con datos guardados antiguos (línea 45)

Este es el cambio menos obvio y el más importante.

Busca: `if (guardado) return JSON.parse(guardado)`

Está así (líneas 42-50):

```
function cargar() {
  try {
    const guardado = localStorage.getItem(CLAVE)
    if (guardado) return JSON.parse(guardado)
  } catch {
    // localStorage no disponible o dato corrupto: se usan las semillas.
  }
  return clonar(semillas())
}
```

Debe quedar así:

```
function cargar() {
  try {
    const guardado = localStorage.getItem(CLAVE)
    // Las semillas van primero: si el navegador guardó una versión anterior sin
    // alguna colección nueva, esa colección igual existe y la maqueta no se cae.
    if (guardado) return { ...clonar(semillas()), ...JSON.parse(guardado) }
  } catch {
    // localStorage no disponible o dato corrupto: se usan las semillas.
  }
  return clonar(semillas())
}
```

Por qué Cualquier persona que ya haya abierto la maqueta tiene guardado en su navegador un objeto `coipo_demo_v1` sin la clave `valoraciones`. Sin este cambio, al desplegar la funcionalidad esos navegadores cargarían el objeto viejo,

`store.valoraciones` sería `undefined` y la app mostraría **pantalla en blanco** con `TypeError: Cannot read properties of undefined (reading 'filter')`.

El operador `...` (*spread*) copia las propiedades de un objeto dentro de otro; el que va a la derecha gana. Así, lo guardado por el usuario se conserva y lo que falta se completa con las semillas.

Verificación del paso 2 La app sigue funcionando igual (todavía no hay nada visible). Para comprobar que la colección existe: `F12` → `Application` → `Local Storage` → `http://localhost:5173` → clave `coipo_demo_v1`. Si ya había datos guardados, verás la clave `valoraciones` recién después de que la app escriba algo; para verla de inmediato usa "Reiniciar demostración".

2.4 Paso 3 — Los endpoints (archivo nuevo)

Crea `frontend/src/api/valoraciones.js`.


```

/**
 * Endpoints de valoraciones (maqueta).
 * Emula: /api/inmuebles/{id}/valoraciones (GET y POST)
 *
 * Regla de negocio: solo puede valorar quien tuvo una reserva finalizada en el
 * inmueble, y una sola vez por reserva.
 */

import { responder, fallar, lista } from './client.js'
import { store, persistir, siguienteId, registrarAuditoria } from './store.js'

const deInmueble = (inmuebleId) =>
  store.valoraciones.filter((v) => v.inmueble_id === Number(inmuebleId))

/**
 * Promedio y total de un inmueble. Es SÍNCRONA a propósito: la usan otros
 * endpoints para adjuntar el resumen sin encadenar promesas.
 */
export function resumenValoracion(inmuebleId) {
  const items = deInmueble(inmuebleId)
  if (items.length === 0) return { promedio: null, total: 0 }
  const suma = items.reduce((acc, v) => acc + v.estrellas, 0)
  return { promedio: Math.round((suma / items.length) * 10) / 10, total: items.length }
}

/** Valoración asociada a una reserva, o null si todavía no se valoró. */
export function valoracionDeReserva(codigo) {
  return store.valoraciones.find((v) => v.reserva_codigo === codigo) ?? null
}

/** GET /api/inmuebles/{id}/valoraciones */
export function getValoraciones(inmuebleId) {
  const items = deInmueble(inmuebleId).sort((a, b) => (a.fecha < b.fecha ? 1 : -1))
  return responder({ ...lista(items), resumen: resumenValoracion(inmuebleId) })
}

/** POST /api/inmuebles/{id}/valoraciones */
export function crearValoracion({ inmueble_id, reserva_codigo, estrellas, comentario }, actor) {
  const nota = Number(estrellas)
  if (!Number.isInteger(nota) || nota < 1 || nota > 5) {
    return fallar(422, 'La valoración debe ser un número entero de 1 a 5 estrellas.')
  }

  const reserva = store.reservas.find((r) => r.codigo === reserva_codigo)
  if (!reserva) return fallar(404, 'Reserva no encontrada')
  if (reserva.estado !== 'finalizada') {
    return fallar(409, 'Solo se pueden valorar estadías finalizadas.')
  }
  if (valoracionDeReserva(reserva_codigo)) {

```

```

    return fallar(409, 'Esta reserva ya fue valorada.')
  }

  const valoracion = {
    id: siguienteId(store.valoraciones),
    inmueble_id: Number(inmuelle_id),
    reserva_codigo,
    usuario_id: actor?.id ?? null,
    autor: actor?.nombre ?? 'Usuario',
    estrellas: nota,
    comentario: comentario?.trim() || null,
    fecha: new Date().toISOString().slice(0, 19),
  }

  store.valoraciones.push(valoracion)
  persistir()

  const inmueble = store.inmuebles.find((i) => i.id === valoracion.inmuelle_id)
  registrarAuditoria({
    usuario: valoracion.autor,
    perfil: actor?.perfil ?? 'Afiliado',
    accion: 'Valoración de estadía',
    entidad: inmueble?.nombre ?? `Inmueble ${valoracion.inmuelle_id}`,
    detalle: `${nota} de 5 estrellas en la reserva ${reserva_codigo}.`,
  })

  return responder(valoracion)
}

```

Cómo leer este archivo

Elemento	Qué hace
<code>responder(datos)</code>	Devuelve una promesa que resuelve en 180 ms, imitando la red
<code>fallar(422, '...')</code>	Devuelve una promesa que rechaza , imitando un error HTTP
<code>422, 404, 409</code>	Códigos HTTP: dato inválido / no existe / conflicto con el estado actual
<code>persistir()</code>	Guarda el store en <code>localStorage</code>
<code>registrarAuditoria(...)</code>	Deja la huella que exige la ley de datos personales
<code>siguienteId(coleccion)</code>	Calcula el próximo <code>id</code> (lo hará la BD en la fase 2)
<code>Math.round(x * 10) / 10</code>	Redondea a un decimal: 4.5, no 4.499999
<code>comentario?.trim() null</code>	Si viene vacío o con espacios, guarda <code>null</code> en vez de ''

Detalle importante: `resumenValoracion` y `valoracionDeReserva` **no** son endpoints: son funciones normales (síncronas) que otros endpoints usan internamente. Los endpoints son los que devuelven `responder(...)` / `fallar(...)`.

Verificación del paso 3 (sin interfaz todavía) En Chrome, `F12` → `Console`, pega:

La ruta empieza con `/` porque en `npm run dev` la aplicación se sirve en la raíz: `vite.config.js` ya no declara ningún `base`, cada destino fija el suyo al construir. Si alguna vez pruebas esto contra el sitio de GitHub Pages, ahí el `import` sí lleva el prefijo del repo (`/coipo_cabania/src/api/...`) — aunque en Pages no existe `src/`, porque se sirve el bundle ya construido.

```
const m = await import('/src/api/valoraciones.js'); m.resumenValoracion(9)
```

Debe imprimir `{promedio: 4.5, total: 2}` (las dos valoraciones sembradas del inmueble 9: 5 y 4 → 4.5).

Prueba también un caso sin datos:

```
const m = await import('/src/api/valoraciones.js'); m.resumenValoracion(1)
```

Debe imprimir `{promedio: null, total: 0}`.

Si falla

- `Failed to fetch dynamically imported module` → la ruta del `import` está mal; el archivo debe llamarse exactamente `valoraciones.js` y estar en `src/api/`.
- `Cannot read properties of undefined (reading 'filter')` → falta el cambio 2.2 (`valoraciones: valoracionesSeed`) o no reiniciaste la demostración.

2.5 Paso 4 — El componente de estrellas (archivo nuevo)

Crea `frontend/src/components/inmuebles/Estrellas.jsx`.

```

/**
 * Valoración con estrellas.
 *
 * `Estrellas` solo muestra (lectura); `SelectorEstrellas` permite elegir una
 * nota. El color nunca es el único indicador: siempre acompaña un texto con el
 * promedio o la nota elegida (guía de accesibilidad, INSUMO/ui_ux.md).
 */

import { useState } from 'react'
import { Star } from 'lucide-react'

const NOTAS = [1, 2, 3, 4, 5]

export function Estrellas({ valor = 0, total = null, tamano = 15, className = '' }) {
  const llenas = Math.round(valor)

  return (
    <span className={`inline-flex items-center gap-1.5 ${className}`}>
      <span className="inline-flex" aria-hidden="true">
        {NOTAS.map((n) => (
          <Star
            key={n}
            size={tamano}
            className={n <= llenas ? 'text-amber-500' : 'text-slate-300'}
            fill={n <= llenas ? 'currentColor' : 'none'}
          />
        ))}
      </span>
      <span className="tabular text-sm text-slate-600">
        {valor.toFixed(1)}
        {total !== null && ` (${total})`}
      </span>
      <span className="sr-only">
        {valor.toFixed(1)} de 5 estrellas
        {total !== null && `, ${total} valoraciones`}
      </span>
    </span>
  )
}

export function SelectorEstrellas({ valor = 0, onCambiar, tamano = 28 }) {
  const [previa, setPrevia] = useState(0)
  const activas = previa || valor

  return (
    <div>
      <div className="flex items-center gap-1" role="group" aria-label="Nota de la estadía">
        {NOTAS.map((n) => (
          <button

```

```

    key={n}
    type="button"
    aria-label={` ${n} ${n === 1 ? 'estrella' : 'estrellas'}`}
    aria-pressed={valor === n}
    onClick={() => onCambiar(n)}
    onMouseEnter={() => setPrevia(n)}
    onMouseLeave={() => setPrevia(0)}
    onFocus={() => setPrevia(n)}
    onBlur={() => setPrevia(0)}
    className="cursor-pointer rounded p-1 transition-transform hover:scale-110"
  >
    <Star
      size={tamano}
      className={n <= activas ? 'text-amber-500' : 'text-slate-300'}
      fill={n <= activas ? 'currentColor' : 'none'}
    />
  </button>
))}
</div>
<p className="mt-1 text-sm text-slate-600" aria-live="polite">
  {valor === 0 ? 'Seleccione una nota de 1 a 5.' : `Nota elegida: ${valor} de 5.`}
</p>
</div>
)
}

```

Decisiones de diseño que conviene entender

Línea / elemento	Por qué
Dos componentes en un archivo	Van siempre juntos y comparten NOTAS . La regla del linter only-export-components permite exportar varios componentes ; lo que no permite es mezclar componentes con constantes exportadas.
<code>const activas = previa valor</code>	Muestra la vista previa mientras el mouse pasa por encima; si no hay previa (vale <code>0</code>), muestra la nota ya elegida.
<code>aria-hidden="true"</code> en las estrellas	Un lector de pantalla no debe leer "estrella estrella estrella..."; lee el texto de al lado.
<code>sr-only</code>	Clase de Tailwind: visible solo para lectores de pantalla.
<code>aria-live="polite"</code>	Anuncia el cambio de nota a quien no ve la pantalla.
<code>type="button"</code>	Obligatorio: sin esto, dentro de un <code><form></code> un <code><button></code> envía el formulario y recarga la página.
<code>onFocus</code> / <code>onBlur</code>	La vista previa también funciona navegando con el teclado (Tab).
<code>valor.toFixed(1)</code>	Fuerza un decimal: se ve 4.0 y no 4 .

Verificación del paso 4 Todavía no se usa en ninguna pantalla, así que **no se ve nada**. Comprueba al menos que el archivo compila: guarda y mira la terminal de `npm run dev`; no debe aparecer ningún error rojo.

2.6 Paso 5 — Adjuntar el promedio a los inmuebles

Archivo: `frontend/src/api/inmuebles.js` — tres cambios.

Cambio 5.1 — import y función auxiliar (línea 8)

Busca: `import { store, persistir, siguienteId, registrarAuditoria } from './store.js'`

Está así (líneas 6-11):

```
import { eachDayOfInterval, parseISO, isWithinInterval, subDays } from 'date-fns'
import { responder, fallar, lista } from './client.js'
import { store, persistir, siguienteId, registrarAuditoria } from './store.js'
import { ESTADOS_QUE_OCUPAN } from '../lib/estados.js'
import { aISO } from '../lib/formato.js'

/** GET /api/inmuebles?region=&tipo=&capacidad_min=&busqueda= */
```

Debe quedar así:

```
import { eachDayOfInterval, parseISO, isWithinInterval, subDays } from 'date-fns'
import { responder, fallar, lista } from './client.js'
import { store, persistir, siguienteId, registrarAuditoria } from './store.js'
import { resumenValoracion } from './valoraciones.js'
import { ESTADOS_QUE_OCUPAN } from '../lib/estados.js'
import { aISO } from '../lib/formato.js'

/** Adjunta el promedio de valoraciones, igual que el backend lo devolvería. */
const conValoracion = (inmueble) => ({
  ...inmueble,
  valoracion: resumenValoracion(inmueble.id),
})

/** GET /api/inmuebles?region=&tipo=&capacidad_min=&busqueda= */
```

Cambio 5.2 — el listado (línea 31)

Busca: `return responder(lista(items))`

Código	
Está	<code>return responder(lista(items))</code>
Queda	<code>return responder(lista(items.map(conValoracion)))</code>

Cambio 5.3 — el detalle (línea 38)

Busca: `return responder(inmueble)`

Código	
Está	<code>return responder(inmueble)</code>

Código

Queda `return responder(conValoracion(inmueble))`

Por qué así El componente **no** debe calcular el promedio: eso es lógica de negocio y vive en la capa `api/`. Cuando exista FastAPI, el backend devolverá el campo `valoracion` ya calculado y la interfaz no cambiará ni una línea.

Verificación del paso 5 F12 → `Console`:

```
const m = await import('/src/api/inmuebles.js'); (await m.getInmueble(9)).valoracion
```

Debe imprimir `{promedio: 4.5, total: 2}`.

Si falla

- The requested module does not provide an export named 'resumenValoracion' → en `valoraciones.js` la función debe estar declarada con `export function`.
- Pantalla en blanco con error de *circular dependency* → asegúrate de que `valoraciones.js` **no** importe nada de `inmuebles.js`. La dependencia va en un solo sentido: `inmuebles.js` → `valoraciones.js`.

2.7 Paso 6 — Mostrar el promedio en el catálogo

Archivo: `frontend/src/components/inmuebles/InmuebleCard.jsx` — **dos cambios.**

Cambio 6.1 — import (línea 6)

Busca: `import { Badge } from '../ui/Badge.jsx'`

Está así (líneas 6-7):

```
import { Badge } from '../ui/Badge.jsx'
import { rutaFoto } from './fotos.js'
```

Debe quedar así:

```
import { Badge } from '../ui/Badge.jsx'
import { Estrellas } from './Estrellas.jsx'
import { rutaFoto } from './fotos.js'
```

Cambio 6.2 — las estrellas bajo el nombre (línea 43-45)

Busca: `{inmueble.nombre}`

Está así (líneas 43-46):

```
<h3 className="text-base leading-snug font-semibold text-verde-900 group-hover:underline">
  {inmueble.nombre}
</h3>
```

Debe quedar así (se agrega el bloque nuevo después del `</h3>`):

```
<h3 className="text-base leading-snug font-semibold text-verde-900 group-hover:underline">
  {inmueble.nombre}
</h3>

{inmueble.valoracion?.total > 0 && (
  <p className="mt-1.5">
    <Estrellas
      valor={inmueble.valoracion.promedio}
      total={inmueble.valoracion.total}
      tamano={14}
    />
  </p>
)}
```

Por qué el guardia `?.total > 0` Si el inmueble no tiene valoraciones, `promedio` es `null` y `Estrellas` haría `null.toFixed(1) → TypeError`. Además, mostrar `0.0 (0)` sería peor que no mostrar nada. La interrogación (`?.`) protege del caso en que `valoracion` no venga en absoluto.

Verificación del paso 6  *primer resultado visible*

1. Ve al catálogo y presiona **"Reiniciar demostración"** (para cargar las semillas nuevas).
2. Busca "Peñuelas" (inmueble 9): la tarjeta muestra ★★★★★ **4.5 (2)** bajo el nombre.
3. Busca "Toconao" (inmueble 1): **no** muestra estrellas. Correcto: no tiene valoraciones.

Si falla

- Ninguna tarjeta muestra estrellas → falta el cambio 5.2 (`items.map(conValoracion)`).
- Todas muestran `0.0 (0)` → escribiste el guardia como `{inmueble.valoracion && (...)}`; un objeto siempre es "verdadero", incluso con `total: 0`.
- `Cannot read properties of null (reading 'toFixed')` → falta el guardia por completo.

2.8 Paso 7 — Valoraciones en la ficha del inmueble

Archivo: [frontend/src/pages/publico/FichaInmueble.jsx](#) — **cinco cambios.**

Cambio 7.1 — imports (líneas 12, 15, 20)

Busca: `import { getInmueble } from '../..api/inmuebles.js'`

Línea	Está	Queda
12	<code>import { getInmueble } from '../..api/inmuebles.js'</code>	igual, y debajo se agrega <code>import { getValoraciones } from '../..api/valoraciones.js'</code>
15	<code>import { pesos } from '../..lib/formato.js'</code>	<code>import { fechaCorta, pesos } from '../..lib/formato.js'</code>
20	<code>import { CalendarioDisponibilidad } from '.../CalendarioDisponibilidad.jsx'</code>	igual, y debajo se agrega <code>import { Estrellas } from '../..components/inmuebles/Estrellas.jsx'</code>

Resultado del bloque de imports (líneas 12-22):


```
import { getInmueble } from '../../api/inmuebles.js'
import { getValoraciones } from '../../api/valoraciones.js'
import { etiquetaTipo, nombreRegion } from '../../fixtures/inmuebles.js'
import { tarifas } from '../../fixtures/tarifas.js'
import { fechaCorta, pesos } from '../../lib/formato.js'
import { useRol } from '../../context/RolContext.jsx'
import { GaleriaFotos } from '../../components/inmuebles/GaleriaFotos.jsx'
import { MapaInmueble } from '../../components/inmuebles/MapaInmueble.jsx'
import { ZonasInteres } from '../../components/inmuebles/ZonasInteres.jsx'
import { CalendarioDisponibilidad } from '../../components/inmuebles/CalendarioDisponibilidad.jsx'
import { Estrellas } from '../../components/inmuebles/Estrellas.jsx'
```

Cambio 7.2 — componente **Valoraciones** (antes de la línea 86)

Busca: `export function FichaInmueble() {`

Agrega **justo antes** de esa línea:

```

/** Comentarios de quienes ya se alojaron en el inmueble. */
function Valoraciones({ inmuebleId }) {
  const [datos, setDatos] = useState(null)

  useEffect(() => {
    let vigente = true
    getValoraciones(inmuebleId).then((r) => vigente && setDatos(r))
    return () => {
      vigente = false
    }
  }, [inmuebleId])

  if (!datos) return <p className="text-sm text-slate-500">Cargando valoraciones...</p>

  if (datos.total === 0) {
    return (
      <p className="text-sm text-slate-500">
        Este inmueble todavía no tiene valoraciones. Se publican cuando las estadías
        finalizan.
      </p>
    )
  }

  return (
    <div>
      <Estrellas valor={datos.resumen.promedio} total={datos.resumen.total} tamano={18} />
      <ul className="mt-3 divide-y divide-arena-200">
        {datos.items.map((v) => (
          <li key={v.id} className="py-3">
            <div className="flex flex-wrap items-center justify-between gap-2">
              <span className="text-sm font-medium text-slate-800">{v.autor}</span>
              <span className="tabular text-xs text-slate-500">{fechaCorta(v.fecha)}</span>
            </div>
            <Estrellas valor={v.estrellas} tamano={13} className="mt-1" />
            {v.comentario && (
              <p className="mt-1.5 text-sm text-slate-600">{v.comentario}</p>
            )}
          </li>
        ))}
      </ul>
    </div>
  )
}

export function FichaInmueble() {

```

Por qué un componente aparte Tiene su propio estado y su propia petición. Si se metiera dentro de `FichaInmueble`, cualquier cambio de estado de la ficha volvería a dibujar también esto. Además, `datos` en `null` distingue "cargando" de

"sin valoraciones" (`total === 0`): **son estados distintos y se ven distinto.**

Es el mismo patrón `let vigente = true ... return () => { vigente = false }` que ya usa el resto del proyecto (ejercicio 10).

Cambio 7.3 — promedio bajo el título (línea 135)

Busca: `<h1 className="mt-2 text-2xl font-semibold text-verde-900">{inmueble.nombre}</h1>`

Está así:

```
<h1 className="mt-2 text-2xl font-semibold text-verde-900">{inmueble.nombre}</h1>
<p className="mt-1 flex items-center gap-1.5 text-sm text-slate-600">
```

Debe quedar así:

```
<h1 className="mt-2 text-2xl font-semibold text-verde-900">{inmueble.nombre}</h1>
{inmueble.valoracion?.total > 0 && (
  <p className="mt-1">
    <Estrellas
      valor={inmueble.valoracion.promedio}
      total={inmueble.valoracion.total}
      tamano={16}
    />
  </p>
)}
<p className="mt-1 flex items-center gap-1.5 text-sm text-slate-600">
```

Cambio 7.4 — la tarjeta de valoraciones (línea 214)

Busca: `<h2 className="text-lg font-semibold text-verde-900">Ubicación y acceso</h2>`

Está así (líneas 214-215):

```
<Tarjeta className="p-5">
  <h2 className="text-lg font-semibold text-verde-900">Ubicación y acceso</h2>
```

Debe quedar así (se agrega una tarjeta completa **antes**):

```
<Tarjeta className="p-5">
  <h2 className="text-lg font-semibold text-verde-900">Valoraciones de huéspedes</h2>
  <p className="mt-1 mb-3 text-sm text-slate-600">
    Opiniones de funcionarios y funcionarias que ya se alojaron en este inmueble.
  </p>
  <Valoraciones inmuebleId={inmueble.id} />
</Tarjeta>

<Tarjeta className="p-5">
  <h2 className="text-lg font-semibold text-verde-900">Ubicación y acceso</h2>
```

Verificación del paso 7

1. Entra a la ficha del inmueble 9 (URL directa: <http://localhost:5173/inmuebles/9>).
2. Bajo el título grande: ★★★★★ 4.5 (2).
3. Más abajo, entre "Descripción" y "Ubicación y acceso", la tarjeta **Valoraciones de huéspedes** con el promedio y **dos** comentarios: Pedro Millán Quezada (5.0, 24 jul 2026) y Ana Villalobos Díaz (4.0, 18 jun 2026), ordenados del más nuevo al más antiguo.
4. Entra al inmueble 1 (</inmuebles/1>): la tarjeta existe pero dice "Este inmueble todavía no tiene valoraciones...", y **no** hay estrellas bajo el título.

Si falla

- `fechaCorta is not defined` → falta el cambio 7.1 en la línea 15.
- `Valoraciones is not defined` → definiste el componente **después** de usarlo dentro de otro archivo, o lo pegaste dentro de la función `FichaInmueble` por error. Debe estar en el nivel superior del archivo.
- La tarjeta aparece pero queda cargando para siempre → `getValoraciones` no resolvió: revisa la consola; probablemente `store.valoraciones` es `undefined` (paso 2).
- Los comentarios salen ordenados al revés → el `sort` compara textos ISO: `(a, b) => (a.fecha < b.fecha ? 1 : -1)` deja el más reciente primero.

2.9 Paso 8 — Saber si una reserva ya fue valorada

Archivo: [frontend/src/api/reservas.js](#) — dos cambios.

Cambio 8.1 — import (línea 9)

Busca: `import { rangoDisponible } from './inmuebles.js'`

	Código
Está	<code>import { rangoDisponible } from './inmuebles.js'</code>
Queda	<code>import { rangoDisponible } from './inmuebles.js'</code> <code>import { valoracionDeReserva } from './valoraciones.js'</code>

Cambio 8.2 — adjuntar la valoración (líneas 31-35)

Busca: `// Se adjunta el inmueble para no obligar a la interfaz a cruzar datos.`

Está así:

```
// Se adjunta el inmueble para no obligar a la interfaz a cruzar datos.
const conInmueble = items.map((r) => ({
  ...r,
  inmueble: store.inmuebles.find((i) => i.id === r.inmueble_id) ?? null,
}))
```

Debe quedar así:

```
// Se adjunta el inmueble y la valoración para no obligar a la interfaz a
// cruzar datos.
const conInmueble = items.map((r) => ({
  ...r,
  inmueble: store.inmuebles.find((i) => i.id === r.inmueble_id) ?? null,
  valoracion: valoracionDeReserva(r.codigo),
}))
```

Verificación del paso 8 F12 → **Console**:

```
const m = await import('/src/api/reservas.js'); (await m.getReservas({ usuario_id: 1 })).items.map((r) => [r.
```

Debe listar las reservas de María Fuentes con **null** en la última columna (aún no ha valorado ninguna).

2.10 Paso 9 — El formulario para valorar

Archivo: [frontend/src/pages/publico/MisReservas.jsx](#) — **seis cambios**. Es el paso más largo.

Cambio 9.1 — imports (líneas 4-5)

Está así:

```
import { CalendarPlus, CircleSlash } from 'lucide-react'
import { anularReserva, getReservas } from '../../api/reservas.js'
```

Debe quedar así:

```
import { CalendarPlus, CircleSlash, Star } from 'lucide-react'
import { anularReserva, getReservas } from '../../api/reservas.js'
import { crearValoracion } from '../../api/valoraciones.js'
```

Cambio 9.2 — más imports (líneas 10-21)

Está así:

```
import {
  FiltroEstados,
  TablaReservas,
} from '../../components/reservas/TablaReservas.jsx'
import { Modal } from '../../components/ui/Modal.jsx'
import {
  Aviso,
  Boton,
  Cargando,
  Tarjeta,
  TituloSeccion,
} from '../../components/ui/Elementos.jsx'
```

Debe quedar así:

```
import {
  FiltroEstados,
  TablaReservas,
} from '../components/reservas/TablaReservas.jsx'
import {
  Estrellas,
  SelectorEstrellas,
} from '../components/inmuebles/Estrellas.jsx'
import { Modal } from '../components/ui/Modal.jsx'
import {
  Aviso,
  Boton,
  Campo,
  Cargando,
  Tarjeta,
  TituloSeccion,
  clasesInput,
} from '../components/ui/Elementos.jsx'
```

(Se agregaron `Campo` y `clasesInput`, que ya existen en `Elementos.jsx` y dan a los formularios el aspecto estándar del sistema.)

Cambio 9.3 — estados nuevos (línea 30-31)

Busca: `const [anulando, setAnulando] = useState(false)`

Está así:

```
const [porAnular, setPorAnular] = useState(null)
const [anulando, setAnulando] = useState(false)
```

Debe quedar así:

```
const [porAnular, setPorAnular] = useState(null)
const [anulando, setAnulando] = useState(false)
const [porValorar, setPorValorar] = useState(null)
const [estrellas, setEstrellas] = useState(0)
const [comentario, setComentario] = useState('')
const [guardando, setGuardando] = useState(false)
const [errorValoracion, setErrorValoracion] = useState(null)
```

Qué hace cada uno

Estado	Para qué
<code>porValorar</code>	La reserva que se está valorando; <code>null</code> = el modal está cerrado
<code>estrellas</code>	La nota elegida (0 = todavía no elige)

Estado	Para qué
<code>comentario</code>	El texto del <code><textarea></code>
<code>guardando</code>	Muestra el spinner y evita doble envío
<code>errorValoracion</code>	Mensaje de error devuelto por la API

Cambio 9.4 — las funciones (antes de la línea 66)

Busca: `const confirmarAnulacion = async () => {`

Agrega **justo antes:**

```
const abrirValoracion = (reserva) => {
  setPorValorar(reserva)
  setEstrellas(0)
  setComentario('')
  setErrorValoracion(null)
}

const enviarValoracion = async () => {
  setGuardando(true)
  setErrorValoracion(null)
  try {
    await crearValoracion(
      {
        inmueble_id: porValorar.inmuelle_id,
        reserva_codigo: porValorar.codigo,
        estrellas,
        comentario,
      },
      actor,
    )
    setPorValorar(null)
    cargar()
  } catch (e) {
    setErrorValoracion(e.detail ?? 'No fue posible registrar la valoración.')
  } finally {
    setGuardando(false)
  }
}
```

Puntos clave

- `abrirValoracion` **limpia** el formulario cada vez. Sin eso, al valorar una segunda reserva aparecería la nota anterior.
- `try / catch / finally`: si la API rechaza (por ejemplo, "ya fue valorada"), se muestra el mensaje; el `finally` apaga el spinner pase lo que pase.
- `e.detail` es el campo de error que define `client.js` (`{ status, detail }`), igual que FastAPI.
- `cargar()` (definida en la línea 33) vuelve a pedir las reservas para que la tabla muestre la nota recién puesta.

- **actor** viene de `useRol()` (línea 26) y es quien queda en la auditoría.

Cambio 9.5 — el botón en la tabla (líneas 123-130)

Busca: `acciones={r} =>`

Está así:

```
acciones={r} =>
  ANULABLES.includes(r.estado) ? (
    <Boton variante="peligro" onClick={() => setPorAnular(r)}>
      <CircleSlash size={14} aria-hidden="true" />
      Desistir
    </Boton>
  ) : null
}
```

Debe quedar así:

```
acciones={r} => {
  if (ANULABLES.includes(r.estado)) {
    return (
      <Boton variante="peligro" onClick={() => setPorAnular(r)}>
        <CircleSlash size={14} aria-hidden="true" />
        Desistir
      </Boton>
    )
  }
  if (r.estado === 'finalizada') {
    return r.valoracion ? (
      <Estrellas valor={r.valoracion.estrellas} tamano={13} />
    ) : (
      <Boton variante="secundario" onClick={() => abrirValoracion(r)}>
        <Star size={14} aria-hidden="true" />
        Valorar
      </Boton>
    )
  }
  return null
}
```

Fíjate en el cambio de forma: de `acciones={r} => {...}` (flecha que devuelve directo) a `acciones={r} => { ... return ... }` (flecha con cuerpo y `return` explícito). Con dos condiciones encadenadas, el `if` se lee mucho mejor que un ternario anidado.

Cambio 9.6 — el modal (después de la línea 180)

Busca: `</Modal>` (el que cierra el modal de desistimiento, casi al final del archivo)

Está así (líneas 172-183):

```
        <p className="text-xs text-slate-500">
          Política vigente: la anulación debe avisarse con al menos{' '}
          ...
        </p>
      </div>
    })
  </Modal>
</>
)
```

Debe quedar así:

```

    <p className="text-xs text-slate-500">
      Política vigente: la anulación debe avisarse con al menos{' '}
      ...
    </p>
  </div>
)}
</Modal>

<Modal
  abierto={Boolean(porValorar)}
  onCerrar={() => setPorValorar(null)}
  titulo="Valorar la estadía"
  descripcion={porValorar ? `${porValorar.codigo} · ${porValorar.inmueble?.nombre}` : ''}
  pie={
    <>
      <Boton variante="neutro" onClick={() => setPorValorar(null)}>
        Cancelar
      </Boton>
      <Boton
        cargando={guardando}
        disabled={estrellas === 0}
        onClick={enviarValoracion}
      >
        Enviar valoración
      </Boton>
    </>
  }
>
  <div className="space-y-4">
    <Campo etiqueta="¿Cómo estuvo la estadía?" requerido>
      <SelectorEstrellas valor={estrellas} onChange={setEstrellas} />
    </Campo>

    <Campo etiqueta="Comentario (opcional)" ayuda="Máximo 300 caracteres.">
      <textarea
        value={comentario}
        onChange={(e) => setComentario(e.target.value)}
        maxLength={300}
        rows={3}
        placeholder="Qué destacaría del inmueble o qué habría que mejorar."
        className={clasesInput}
      />
    </Campo>

    {errorValoracion && <Aviso tono="rojo">{errorValoracion}</Aviso>}

    <p className="text-xs text-slate-500">
      Su valoración queda visible en la ficha del inmueble junto a su nombre.
    </p>

```

```

    </div>
  </Modal>
</>
)
}

```

Detalles

- `abierto={Boolean(porValorar)}` convierte el objeto (o `null`) en `true/false`.
- `disabled={estrellas === 0}` implementa la validación en la interfaz; la API valida otra vez por su cuenta. **Ambas validaciones son necesarias:** la de interfaz para guiar, la de API porque es la que manda.
- `<textarea value={...} onChange={...}>` es un componente controlado (ejercicio 9).
- El `<.../>` del `pie` es un **fragmento**: agrupa dos botones sin agregar un `<div>`.

2.11 Paso 10 — Pruebas

10.1 Prueba manual completa (guion exacto)

Antes de empezar: "**Reiniciar demostración**" en el pie de página, y perfil **Afiliado/a** en el selector del encabezado.

#	Acción	Resultado esperado
1	Ir a Catálogo	Las tarjetas con valoraciones muestran estrellas + promedio; las demás no muestran nada
2	Buscar "Peñuelas" y entrar a la ficha	Bajo el título: 4.5 (2) . Tarjeta "Valoraciones de huéspedes" con 2 comentarios
3	Entrar a <code>/inmuebles/1</code>	Mensaje "Este inmueble todavía no tiene valoraciones"
4	Menú "Mis reservas"	Las 3 reservas Finalizadas muestran botón Valorar ; la Confirmada muestra Desistir
5	Clic en Valorar de <code>R-2026-0012</code>	Se abre el modal "Valorar la estadía · R-2026-0012 · Casa de Huéspedes Valdivia"
6	Sin elegir nota	"Enviar valoración" está gris y deshabilitado ; el texto dice "Seleccione una nota de 1 a 5."
7	Pasar el mouse por las estrellas	Se pintan de ámbar en vista previa; al sacar el mouse vuelven
8	Clic en la 4.ª estrella	4 estrellas ámbar y el texto "Nota elegida: 4 de 5." El botón se habilita
9	Escribir un comentario	Se escribe normalmente; al llegar a 300 caracteres no deja seguir
10	Clic en Enviar valoración	El modal se cierra; la fila <code>R-2026-0012</code> ahora muestra ★★★★★ 4.0 en vez del botón
11	Volver a Catálogo y entrar a "Casa de Huéspedes Valdivia"	Aparece tu valoración con tu nombre (María Fuentes Rojas) y la fecha de hoy
12	Cambiar el perfil a Administrador → menú Auditoría	Primera fila: "Valoración de estadía", con el inmueble y "4 de 5 estrellas en la reserva R-2026-0012"

#	Acción	Resultado esperado
13	Recargar la página completa (F5) y volver a "Mis reservas"	La valoración sigue ahí (se guardó en <code>localStorage</code>)
14	Navegar con el teclado: <code>Tab</code> hasta el botón Valorar, <code>Enter</code> , luego <code>Tab</code> por las estrellas y <code>Enter</code>	Todo operable sin mouse; el foco se ve con contorno verde

Los nombres de reserva e inmueble del paso 5 y 10 son los que aparecen realmente: la primera reserva finalizada de María Fuentes es `R-2026-0012`, Casa de Huéspedes Valdivia.

10.2 Casos de error (hay que probarlos también)

Caso	Cómo provocarlo	Resultado esperado
Reserva ya valorada	En la consola: <code>const m = await import('/src/api/valoraciones.js'); m.crearValoracion({inmueble_id:9, reserva_codigo:'R-2026-0006', estrellas:5}, {nombre:'Prueba'}).catch(e => console.log(e))</code>	<code>{status: 409, detail: 'Esta reserva ya fue valorada.'}</code>
Nota fuera de rango	Igual, con <code>estrellas: 9</code> y un código no valorado	<code>{status: 422, detail: 'La valoración debe ser un número entero de 1 a 5 estrellas.'}</code>
Reserva no finalizada	Igual, con <code>reserva_codigo: 'R-2026-0004'</code> (confirmada)	<code>{status: 409, detail: 'Solo se pueden valorar estadías finalizadas.'}</code>
Reserva inexistente	<code>reserva_codigo: 'R-2026-9999'</code>	<code>{status: 404, detail: 'Reserva no encontrada'}</code>

10.3 Controles automáticos

```
cd frontend
npm run lint
npm run build
```

- `lint` debe mostrar **solo los 3 warnings preexistentes** (`graficos.jsx`, `RolContext.jsx`, `AppLayout.jsx`). Cualquier otro es tuyo.
- `build` debe terminar en `✓ built in ...s`.

10.4 Prueba en móvil

F12 → ícono de celular (`Ctrl+Shift+M`) → elige "iPhone SE" (375 px): la tabla de "Mis reservas" debe poder desplazarse y el modal debe caber en pantalla. Ningún elemento debe provocar desplazamiento horizontal de la página.

2.12 Paso 11 — Posibles fallas y cómo resolverlas

Síntoma	Causa más probable	Solución
Pantalla en blanco; consola dice <code>Cannot read properties of undefined (reading 'filter')</code>	Falta <code>valoraciones</code> : <code>valoracionesSeed</code> en <code>semillas()</code> (cambio 2.2), o el navegador tiene guardada la versión antigua del store	Aplica el cambio 2.2 y el 2.3, y presiona "Reiniciar demostración"
<code>Cannot read properties of null (reading 'toFixed')</code>	Se llamó a <code><Estrellas valor={null} /></code> porque falta el guardia <code>?.total > 0</code>	Agrega el guardia (cambios 6.2 / 7.3)
Las estrellas no aparecen en el catálogo pero sí en la ficha	Falta el cambio 5.2 (<code>items.map(conValoracion)</code>)	Aplicalo
El botón "Valorar" no aparece nunca	La reserva no está en estado <code>finalizada</code> , o falta el cambio 9.5	Filtra por "Finalizadas" en la barra de filtros para confirmar
Al valorar, la tabla no se actualiza	Falta <code>cargar()</code> después del <code>await</code> , o falta el cambio 8.2 (la reserva no trae <code>valoracion</code>)	Revisa <code>enviarValoracion</code> y <code>api/reservas.js</code>
El modal se abre con la nota de la valoración anterior	Falta limpiar el estado en <code>abrirValoracion</code>	Agrega <code>setEstrellas(0)</code> y <code>setComentario('')</code>
Se crean valoraciones duplicadas al hacer doble clic	Falta <code>cargando={guardando}</code> en el botón (deshabilita mientras envía)	Agrégalo; la API igual rechaza la segunda con 409
La página se recarga al hacer clic en una estrella	Falta <code>type="button"</code> en el <code><button></code> del selector	Agrégalo
<code>The requested module does not provide an export named 'X'</code>	El nombre importado no coincide con el exportado, o falta <code>export</code>	Compara letra por letra; JavaScript distingue mayúsculas
<code>Failed to resolve import './Estrellas.jsx'</code>	Ruta o nombre de archivo equivocados	Desde <code>components/inmuebles/</code> es <code>./Estrellas.jsx</code> ; desde <code>pages/publico/</code> es <code>../../components/inmuebles/Estrellas.jsx</code>
<code>Warning Each child in a list should have a unique "key" prop</code>	Falta <code>key</code> en algún <code>.map()</code>	Usa <code>key={v.id}</code>
<code>npm run build</code> falla pero <code>npm run dev</code> funcionaba	Import no usado o error que Vite tolera en desarrollo	Lee el mensaje: dice archivo y línea
Todo funciona pero al recargar se pierde	Falta <code>persistir()</code> después de modificar el store	Ya está en <code>crearValoracion</code> ; verifica que no lo hayas borrado

Regla general de depuración: cuando algo no se ve, sigue el dato hacia atrás por la cadena `fixture` → `store` → `api` → `página` → `componente` y haz `console.log` en cada eslabón. El primero que imprima algo distinto de lo esperado es el que falla.

2.13 Paso 12 — Commits

Un commit es una foto del trabajo con un mensaje. **No hagas un solo commit gigante**: divide por paso lógico, así el revisor puede seguir el razonamiento.

En GitHub Desktop, pestaña **Changes**: cada archivo tiene una casilla. **Marca solo los archivos del commit que estás haciendo**, escribe el mensaje abajo a la izquierda y presiona **Commit to feat/valoraciones-estrellas**.

Secuencia sugerida

#	Archivos marcados	Mensaje del commit
1	<code>fixtures/valoraciones.seed.js, api/store.js</code>	Agrega la colección de valoraciones al store de la maqueta
2	<code>api/valoraciones.js</code>	Agrega los endpoints de valoraciones con sus reglas de negocio
3	<code>components/inmuebles/Estrellas.jsx</code>	Agrega el componente de estrellas (lectura y selección)
4	<code>api/inmuebles.js,</code> <code>components/inmuebles/InmubleCard.jsx</code>	Muestra el promedio de valoraciones en el catálogo
5	<code>pages/publico/FichaInmuble.jsx</code>	Agrega la sección de valoraciones a la ficha del inmueble
6	<code>api/reservas.js, pages/publico/MisReservas.jsx</code>	Permite valorar las estadías finalizadas desde Mis reservas

Cómo se escribe un mensaje de commit en este proyecto (mira `git log`): en español, en tiempo presente, describiendo **qué hace el cambio**, no qué archivo tocaste.

✅ **Agrega la sección de valoraciones a la ficha del inmueble** ❌ **cambios, fix, modifica FichaInmuble.jsx, avance del día**

Cuando termines: botón **Push origin** (sube los commits a GitHub).

2.14 Paso 13 — Pull Request

Qué es: la solicitud formal de incorporar tu rama a `main`. Es donde ocurre la revisión de código.

1. GitHub Desktop → botón **Create Pull Request** (abre el navegador). Si no aparece, ve a [GitHub.com](https://github.com) → repositorio → pestaña **Pull requests** → **New pull request**.
2. Verifica arriba: `base: main` ← `compare: feat/valoraciones-estrellas`.
3. Título: **Sistema de valoración de estadías con 5 estrellas**
4. Descripción (plantilla que puedes copiar):

Qué hace

Permite que quien tuvo una estadía finalizada valore el inmueble con 1 a 5 estrellas y un comentario opcional. El promedio se muestra en el catálogo y en la ficha del inmueble.

Por qué

Da a los afiliados información de otros funcionarios antes de reservar y entrega a Bienestar una señal de qué inmuebles necesitan mantención.

Cómo probarlo

1. "Reiniciar demostración" en el pie de página.
2. Catálogo → los inmuebles con valoraciones muestran estrellas y promedio.
3. Ficha del inmueble 9 → sección "Valoraciones de huéspedes" con 2 comentarios.
4. Mis reservas (perfil Afiliado/a) → botón "Valorar" en las reservas finalizadas.
5. Valorar con 4 estrellas → la nota reemplaza al botón y aparece en la ficha del inmueble.
6. Perfil Administrador → Auditoría → queda registrada la acción "Valoración de estadía".

Decisiones de diseño

- La lógica (promedio, validaciones) vive en ``api/``, no en los componentes: cuando exista el backend FastAPI solo se reemplaza esa capa.
- ``api/store.js`` ahora fusiona las semillas con lo guardado en `localStorage`, para que los navegadores con datos de una versión anterior no se caigan al faltarles la colección nueva.
- Una valoración por reserva (no por inmueble), validado en la API con código 409.

Verificación

- [x] ``npm run lint`` sin errores nuevos (se mantienen los 3 warnings preexistentes)
- [x] ``npm run build`` correcto
- [x] Probado en Chrome escritorio y en vista móvil (375 px)
- [x] Navegable con teclado y con textos alternativos para lectores de pantalla

Pendiente / fuera de alcance

- Moderación o eliminación de comentarios por parte de Bienestar.
- Definición con Bienestar de si la valoración debe ser anónima.

5. A la derecha, en **Reviewers**, agrega a quien corresponda del equipo.

6. **Create pull request**.

Qué esperar después

- El revisor deja comentarios en líneas específicas. Responder cada uno: corregir, o explicar por qué se hizo así. **Los comentarios son sobre el código, no sobre ti.**
- Para corregir: edita en tu rama, haz commit y **Push origin**. El PR se actualiza solo.

- Cuando esté aprobado, se fusiona con **Merge pull request**. Ahí se dispara **pages.yml**, que corre **lint + build** y publica la maqueta.
- Después de fusionar: GitHub ofrece **Delete branch**. Acepta. En GitHub Desktop vuelve a **main** y presiona **Fetch origin** para traer el trabajo ya fusionado.

Si el PR muestra archivos que no esperabas Casi siempre es porque la rama se creó desde otra rama y no desde **main**, o porque alguien fusionó cambios a **main** mientras trabajabas. Solución: en GitHub Desktop, **Branch** → **Update from main**.

2.15 Extensiones naturales de esta funcionalidad

Si quieres seguir practicando sobre lo construido (cada uno es un PR aparte):

1. Que la encargada regional vea las valoraciones de su región en el panel regional.
2. Un gráfico de promedio por inmueble en Reportes de Oficina Central (ya se usa **recharts** en **graficos.jsx**).
3. Permitir editar la valoración dentro de los 7 días siguientes.
4. Marcar en el catálogo los inmuebles con promedio bajo 3.0 para priorizar mantención.
5. Filtro "solo inmuebles con 4 estrellas o más" (combina el ejercicio 9 con esta funcionalidad).

Anexo A — Glosario mínimo

Término	Qué es
Componente	Función de JavaScript que devuelve JSX (un pedazo de interfaz)
JSX	Sintaxis parecida a HTML dentro de JavaScript
Props	Parámetros que un componente recibe de su padre
Estado (useState)	Variable que, al cambiar, hace que el componente se vuelva a dibujar
Efecto (useEffect)	Código que corre después de dibujar; sirve para pedir datos
Hook	Función de React que empieza con use . Solo se llaman dentro de componentes y siempre en el mismo orden
Contexto	Dato global disponible en todo el árbol sin pasarlo por props
Promesa / async / await	Manejo de resultados que llegan después (una petición de red)
Fixture	Archivo con datos de mentira que imitan lo que dará la base de datos
Store	Estado en memoria que hace de base de datos en la maqueta
Endpoint	Función de api/ que imita una URL del futuro backend
Tailwind	Librería de estilos por clases utilitarias (p-4 , text-sm)
Lint	Revisión automática de reglas de código
Build	Compilación de la versión optimizada para publicar
Rama (branch)	Línea de trabajo paralela a main
Commit	Foto del trabajo con un mensaje
Pull Request (PR)	Solicitud de fusionar una rama, con revisión de por medio

Término	Qué es
CI	Automatización que corre lint/build/publicación al fusionar

Anexo B — Errores frecuentes y su significado

Mensaje	Significado	Qué revisar
<code>X is not defined</code>	Usaste algo sin importarlo o mal escrito	Los <code>import</code> del principio del archivo
<code>Cannot read properties of undefined (reading 'y')</code>	Intentaste leer <code>y</code> de algo que no existe	Usa <code>objeto?.y</code> ; revisa que el dato llegue
<code>Cannot read properties of null (reading 'y')</code>	Lo mismo, pero el valor es explícitamente <code>null</code>	Agrega un guardia antes de usarlo
<code>Objects are not valid as a React child</code>	Pusiste un objeto donde iba texto	Muestra una propiedad (<code>obj.nombre</code>), no el objeto
<code>Each child in a list should have a unique "key" prop</code>	Falta <code>key</code> en un <code>.map()</code>	Usa un identificador único, no el índice si la lista cambia
<code>Too many re-renders</code>	Cambias estado durante el dibujado	El <code>onClick</code> debe ser <code>() => setX(...)</code> , con flecha
<code>Rendered more hooks than during the previous render</code>	Un <code>useState/useEffect</code> dentro de un <code>if</code> o un bucle	Los hooks van siempre arriba, sin condiciones
<code>Unterminated JSX contents</code>	Falta cerrar una etiqueta	Busca la etiqueta abierta cerca de la línea indicada
<code>Failed to resolve import</code>	Ruta de archivo equivocada	Cuenta los <code>../</code> desde la carpeta del archivo actual
<code>useRol debe usarse dentro de RolProvider</code>	Llamaste <code>useRol()</code> fuera de un componente	Muévelo a un componente dentro de <code><RolProvider></code>
<code>npm error code ENOENT ... package.json</code>	Estás en la carpeta equivocada	<code>cd frontend</code>
<code>Port 5173 is already in use</code>	Ya tienes otro <code>npm run dev</code> corriendo	Cierra la otra terminal, o usa el puerto que ofrezca

Dónde seguir

- [docs/ALCANCE.md](#) — qué entra y qué no en esta fase.
- [docs/GUION_DEMO.md](#) — recorrido de la maqueta perfil por perfil. **Léelo completo antes de tocar código**: es la forma más rápida de entender el negocio.
- [CLAUDE.md](#) — contexto del proyecto y las preguntas abiertas con Bienestar.
- [INSUMO/](#) — requisitos originales. Ante una duda funcional, el PDF de la solicitud manda.
- [frontend/qa/](#) — ejemplos de pruebas automatizadas con navegador real (Puppeteer). Útiles como referencia de cómo se verifica de verdad una funcionalidad.